

Red Neuronal aplicada a la Enfermedad de Carrión

Explicación del código, paso a paso (material de exposición)

Inteligencia Artificial · UNTELS · Caso: enfermedad de Carrión (MINSA) · Comparativa Red Neuronal vs. Árbol de Decisión

Este documento explica, **bloque por bloque**, el código del modelo de **Red Neuronal** que clasifica la fase de la enfermedad de Carrión en **AGUDA** (0) o **ERUPTIVA** (1), usando datos de vigilancia del MINSA (2000–2024, ~46 mil registros). Una red neuronal aprende ajustando los **pesos** de sus conexiones para minimizar el error de predicción.

1. Librerías y carga de datos

```
import pandas as pd, numpy as np
training = pd.read_csv("comparador-ml/data/carrion-train.csv")
```

pandas carga el CSV de entrenamiento en una tabla (DataFrame). **numpy** se usa para las operaciones numéricas (matrices). Trabajamos con el 80% de los datos reservado para entrenar; el otro 20% (prueba) se usa al final para medir qué tan bien generaliza el modelo a casos nuevos.

2. Preprocesamiento: convertir texto a números

Una red neuronal solo entiende números, así que transformamos cada columna:

```
def edad_anios(e, t):
    # tipo_edad: A=años, M=meses, D=días
    t = str(t).upper()
    return e/12 if t=="M" else (e/365 if t=="D" else float(e))

df["edad_anios"] = df.apply(lambda r: edad_anios(r["edad"], r["tipo_edad"]), axis=1)
df["sexo_cod"] = df["sexo"].apply(lambda s: 0 if str(s).upper()=="M" else 1)
df["depto_cod"] = df["departamento"].apply(lambda d: depto_idx[d]) # 0..18
df["target"] = df["enfermedad"].map({"...AGUDA":0, "...ERUPTIVA":1})
```

• **edad_anios**: unifica la edad a años (un bebé de 6 meses → 0.5). • **sexo_cod**: Masculino=0, Femenino=1. • **depto_cod**: cada departamento recibe un número (0 a 18). • **target**: la variable que queremos predecir, convertida a 0/1.

```
columns = ["edad_anios", "sexo_cod", "depto_cod", "ano", "semana"]
x_input = df[columns].values # 5 columnas de entrada (features)
y_target = df["target"].values # la respuesta correcta (0 o 1)
```

x_input son las 5 variables que el modelo recibe; **y_target** es la respuesta correcta de cada caso, con la que el modelo compara sus predicciones durante el entrenamiento.

3. Normalización (paso clave de la red)

```
x_min = x_input.min(axis=0); x_max = x_input.max(axis=0)
rango = np.where((x_max-x_min)==0, 1, x_max-x_min)
x_input_n = (x_input - x_min) / rango # cada columna queda entre 0 y 1
```

Las 5 variables tienen escalas muy distintas: el **año** ronda 2000, la **semana** va de 1 a 53 y la **edad** de 0 a 99. Si no se ajustan, el año (números enormes) dominaría el aprendizaje. La **normalización min-max** lleva cada columna al rango [0, 1] para que todas pesen parejo. (El Árbol de Decisión NO necesita este paso; la red sí.)

4. Arquitectura de la red: 5 → 32 → 32 → 1

```
import keras
from keras import layers
model = keras.Sequential()
model.add(layers.Dense(32, input_dim=5, activation="relu")) # capa oculta 1
model.add(layers.Dense(32, activation="relu", name="layer1")) # capa oculta 2
model.add(layers.Dense(1, activation="sigmoid", name="layer2")) # salida
```

Sequential apila capas una tras otra. **input_dim=5**: entran las 5 variables. Dos capas ocultas de **32 neuronas** con activación **ReLU** (deja pasar lo positivo y anula lo negativo; permite aprender relaciones no lineales). La capa final tiene **1 neurona** con **sigmoid**, que devuelve una probabilidad entre 0 y 1 (probabilidad de ERUPTIVA).

Es exactamente la misma arquitectura que el ejemplo del Titanic del profesor; solo cambian los datos.

5. Compilación y entrenamiento

```
model.compile(loss="binary_crossentropy", optimizer="adam", metrics=["accuracy"])
model.fit(x_input_n, y_target, epochs=400, batch_size=256, verbose=0)
```

loss = binary_crossentropy: la función de error apropiada para clasificación de 2 clases (penaliza fuerte equivocarse con seguridad). **optimizer = adam**: el algoritmo que ajusta los pesos para reducir el error. **epochs = 400**: el modelo recorre todos los datos 400 veces. **batch_size = 256**: procesa de a 256 casos por paso (acelera el entrenamiento porque el dataset es grande; el Titanic, con 915 filas, no lo necesitaba).

6. Evaluación del modelo

```
score = model.evaluate(x_input_n, y_target, verbose=0)
print("Accuracy:", score[1]*100)

from sklearn.metrics import confusion_matrix, classification_report
y_sim = (model.predict(x_input_n).ravel() >= 0.5).astype(int)
print(confusion_matrix(y_target, y_sim))
print(classification_report(y_target, y_sim, target_names=["AGUDA", "ERUPTIVA"]))
```

accuracy: porcentaje de aciertos. La **matriz de confusión** muestra aciertos y errores por clase (cuántas AGUDA predijo bien, cuántas confundió con ERUPTIVA, etc.). El **classification_report** añade precisión, recall y F1. La regla **>= 0.5** convierte la probabilidad de la red en una decisión: 0.5 o más → ERUPTIVA, si no → AGUDA.

7. Predicción de un caso nuevo

```
caso = np.array([[20, 0, 1, 2004, 31]])      # edad, sexo, depto, año, semana
caso_n = (caso - x_min) / rango              # se normaliza igual que el train
prob = model.predict(caso_n)[0][0]          # probabilidad de ERUPTIVA
```

Para predecir un caso nuevo hay que normalizarlo con **los mismos** min/max del entrenamiento. Por eso esos valores se guardan en **norm.json** y se reutilizan en la página web, garantizando que el navegador calcule igual que Python.

8. Guardado y uso en la web

```
model.to_json()                -> mimodelocarrion.json      (estructura)
model.save_weights(...)        -> mimodelocarrion.weights.h5 (pesos)
tensorflowjs ...               -> public/models/carrion/rn/ (formato navegador)
```

El modelo entrenado se guarda y se convierte a **TensorFlow.js**, que es lo que carga la página web. Son los **mismos pesos**: la web no reentrena, solo usa el modelo ya aprendido.

Resultado e interpretación

La red obtiene aproximadamente **72% en entrenamiento y 72% en prueba**. Lo importante: ambos números son casi iguales, lo que indica que la red **generaliza bien** (no memoriza). Da probabilidades prudentes en lugar de certezas absolutas — su gran ventaja frente al Árbol de Decisión, que se compara en el otro documento.